Spam / Ham SMS Classifier

TF-IDF + Multinomial Naive Bayes

Before running: download `spam.csv` from the Kaggle *SMS Spam Collection* dataset. Then run the cells top to bottom (Runtime → Run all).

1. Imports

```
import re
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, f1_score,
                             precision_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

2. Upload dataset

Click **Choose files** and select `spam.csv`.

```
from google.colab import files
uploaded = files.upload()
```

Choose Files spam.csv

spam.csv(text/csv) - 480378 bytes, last modified: 10/22/2026 - 100% done

```
Saving spam.csv to spam.csv
```

3. Load and clean data

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " url ", text)
    text = re.sub(r"\d+", " num ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

df = pd.read_csv("spam.csv", encoding="latin-1")[["v1", "v2"]]
df.columns = ["label", "message"]
df = df.drop_duplicates().dropna()
df["label"] = df["label"].map({"ham": 0, "spam": 1})
df["clean"] = df["message"].apply(clean_text)

print("Dataset shape:", df.shape)
print(df["label"].value_counts())
df.head()
```

```
Dataset shape: (5171, 3)
label
0    4518
1     653
Name: count, dtype: int64
```

	label	message	clean
0	0	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...
1	0	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in num a wkly comp to win fa cup fi...
3	0	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
4	0	Nah I don't think he goes to usf, he lives aro...	nah i don t think he goes to usf he lives arou...

Next steps: Generate code with df | New interactive sheet

4. Train the model

```
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df["label"], test_size=0.2, random_state=42, stratify=df["label"])

model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", ngram_range=(1, 2))),
    ("nb", MultinomialNB(alpha=0.1)),
])
model.fit(X_train, y_train)
print("Training complete")
```

```
Training complete
```

5. Evaluate

```
pred = model.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test, pred):.4f}")
print(f"Precision: {precision_score(y_test, pred):.4f}")
print(f"Recall   : {recall_score(y_test, pred):.4f}")
print(f"F1-score : {f1_score(y_test, pred):.4f}\n")
print(classification_report(y_test, pred, target_names=["ham", "spam"]))

ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=["ham", "spam"], cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

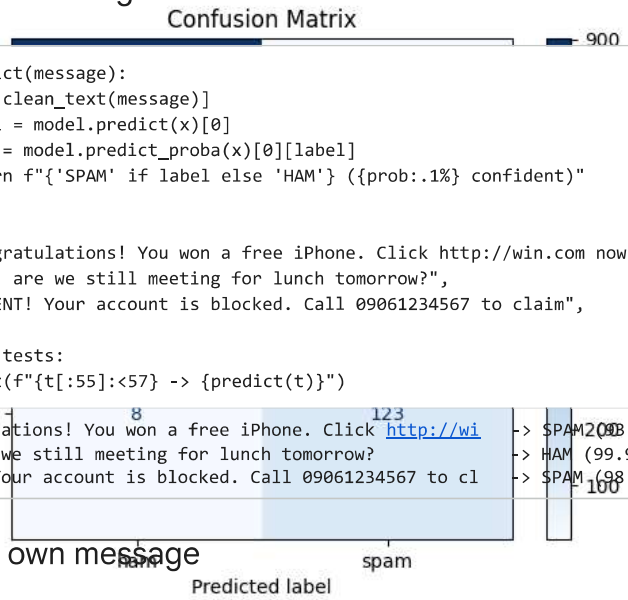
```
Accuracy : 0.9894
Precision: 0.9762
Recall   : 0.9389
F1-score : 0.9572

              precision    recall  f1-score   support

         ham       0.99      1.00      0.99       904
        spam       0.98      0.94      0.96       131

    accuracy                           0.99      1035
   macro avg       0.98      0.97      0.98      1035
weighted avg       0.99      0.99      0.99      1035
```

6. Test with messages




```
def predict(message):
    x = [clean_text(message)]
    label = model.predict(x)[0]
    prob = model.predict_proba(x)[0][label]
    return f"{'SPAM' if label else 'HAM'} ({prob:.1%} confident)"

tests = [
    "Congratulations! You won a free iPhone. Click http://win.com now",
    "Hey, are we still meeting for lunch tomorrow?",
    "URGENT! Your account is blocked. Call 09061234567 to claim",
]
for t in tests:
    print(f"{t[:55]:<57} -> {predict(t)}")
```

```
Congratulations! You won a free iPhone. Click http://wi   -> SPAM (98.4% confident)
Hey, are we still meeting for lunch tomorrow?             -> HAM (99.9% confident)
URGENT! Your account is blocked. Call 09061234567 to cl   -> SPAM (98.5% confident)
```

7. Try your own message

```
my_message = "Type your message here"
print(predict(my_message))
```

```
HAM (93.7% confident)
```

8. Download the trained model (optional)

```
import joblib
joblib.dump(model, "spam_model.joblib")
files.download("spam_model.joblib")
```